

Shortest Paths and Dijkstra's Algorithm

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**

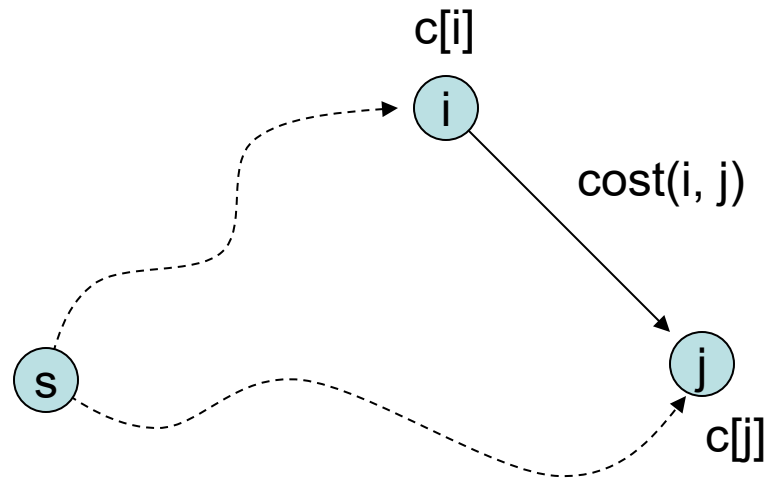


Single-Source Shortest Paths: Algorithms

- Easy cases that we can solve in linear time:
 - Unweighted graphs: use **breadth-first search**.
 - Directed acyclic graphs (DAGs): use **dynamic programming**.
- If edge costs are nonnegative, we'll use **Dijkstra's algorithm**.
- If some edge costs are negative, we use the **Bellman-Ford algorithm**.
- If there is a negative cycle, we are out of luck!

The Triangle Inequality

- Shortest path costs satisfy the triangle inequality:
 $c[i] + \text{cost}(i, j) \geq c[j]$ for every edge (i, j) .



- **Shortest path optimality conditions:** If we have a set of valid shortest path cost labels $c[j]$ that satisfy the triangle inequality for every edge, then these costs must be optimal.

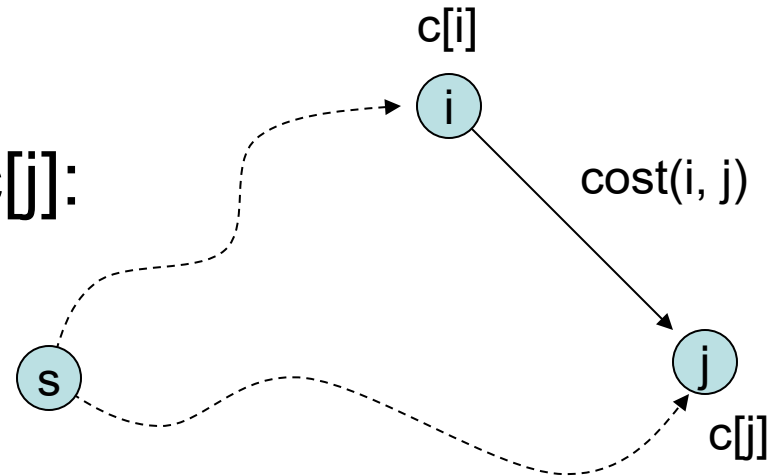
Fixing the Triangle Inequality

- If we ever notice an edge (i, j) that violates the triangle inequality:

$$c[i] + \text{cost}(i, j) < c[j]$$

then we ought to fix things by reducing $c[j]$:

```
Tighten(i, j):  (also called Relax(i, j))  
  If  $c[i] + \text{cost}(i, j) < c[j]$ ,  
     $c[j] = c[i] + \text{cost}(i, j)$   
     $\text{pred}[j] = i$ 
```



- Most shortest path algorithms start with $c[s] = 0$ and $c[j \neq s] = +\infty$, and repeatedly tighten (relax) edges violating the Δ inequality.
 - This guarantees a set of valid cost labels $c[j]$ that will be optimal when we terminate.

Dijkstra's Algorithm

Dijkstra:

`c[s] = 0, c[j ≠ s] = Infinity // often n times largest edge cost`

`Build a priority queue Q on the c[j]'s`

`While Q is nonempty:`

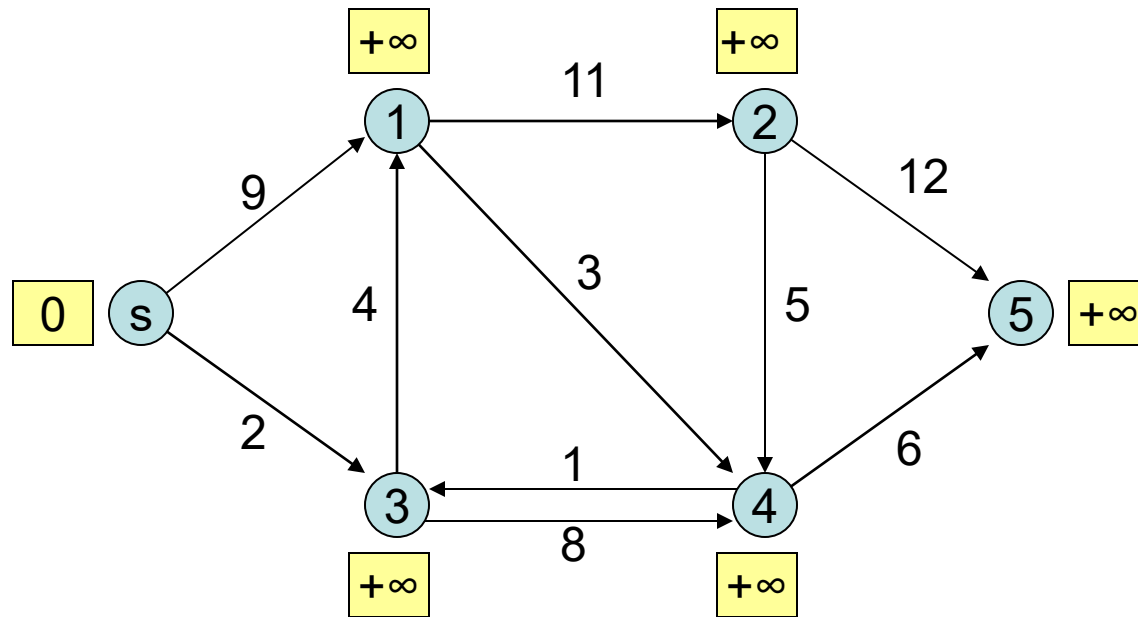
`Remove from Q the node i having minimum label c[i]`

`(at this point we know c[i] is correct)`

`Tighten all edges emanating from i`

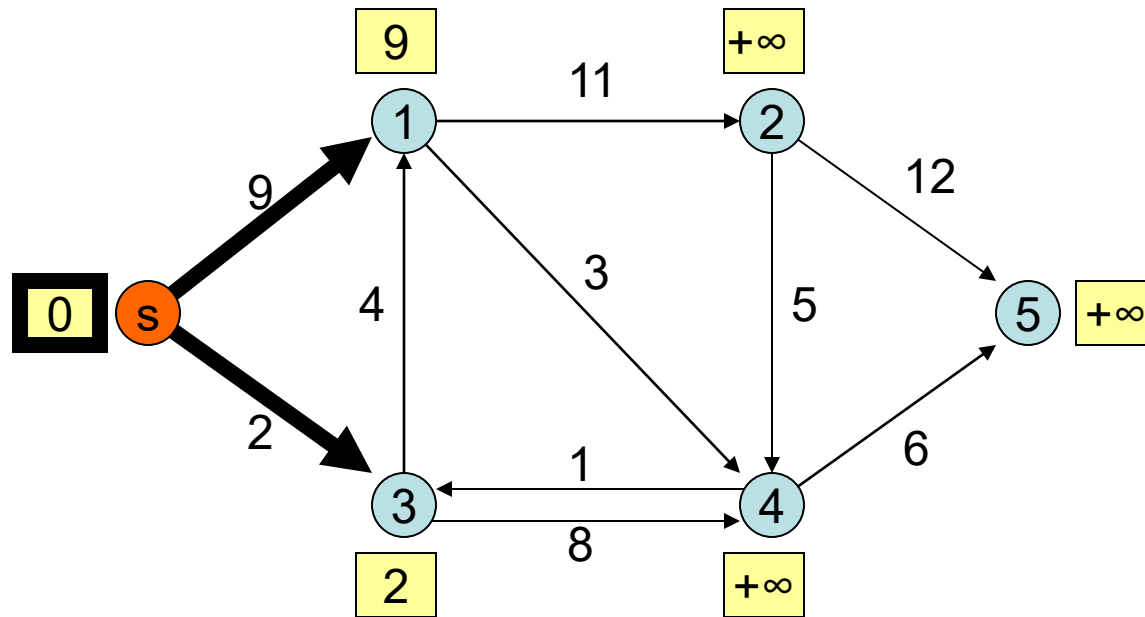
- Only works if edge costs are nonnegative.
- Running time depends on priority queue:
 - n inserts (during initialization)
 - n remove-mins
 - at most m decrease-keys
- Using a binary heap, all these operations run in $O(\log n)$ time, so total is $O(m \log n)$.

Dijkstra's Algorithm – Example



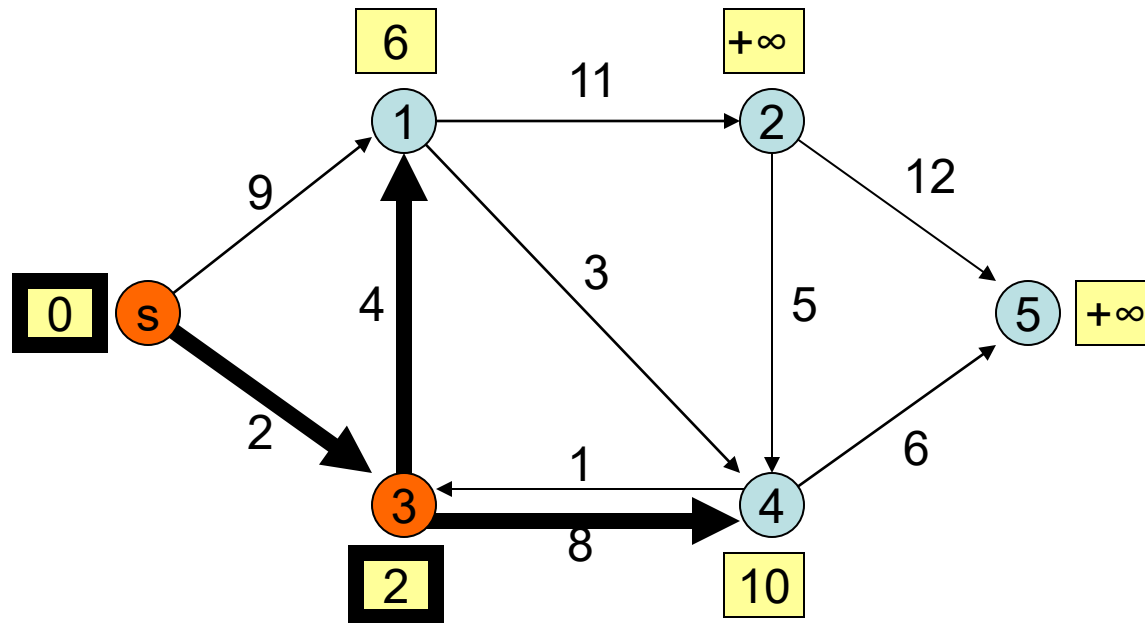
Initialization.

Dijkstra's Algorithm – Example



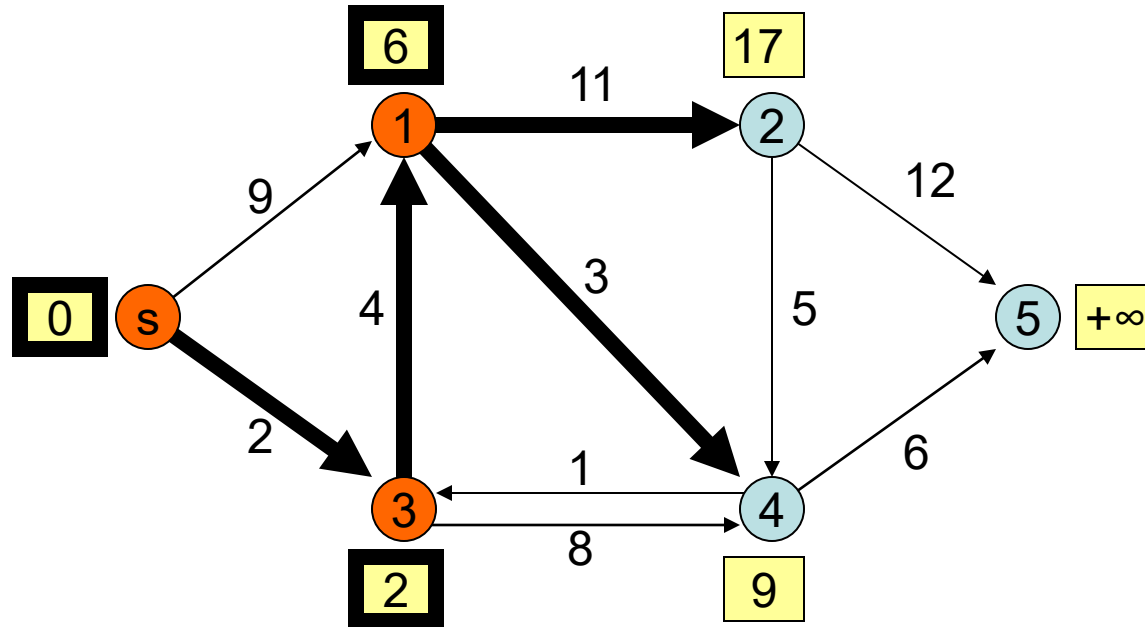
Examine the source node **s**, so **s** becomes “permanently” labeled.
Tighten all edges leaving **s**.

Dijkstra's Algorithm – Example



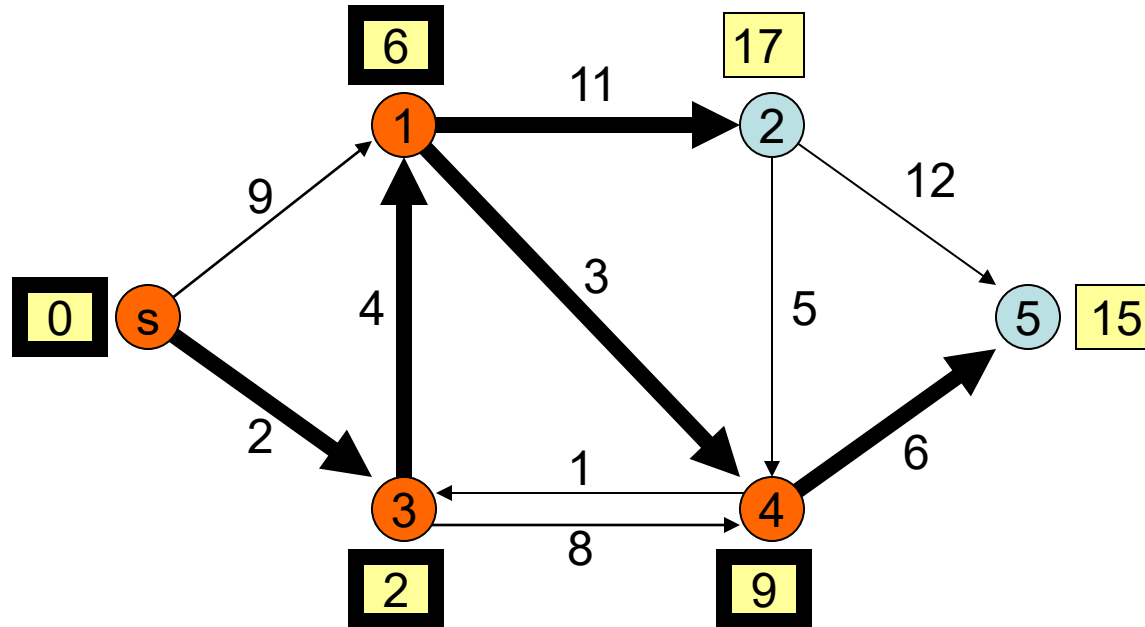
Examine node 3, which becomes permanently labeled.
Tighten all edges leaving node 3.

Dijkstra's Algorithm – Example



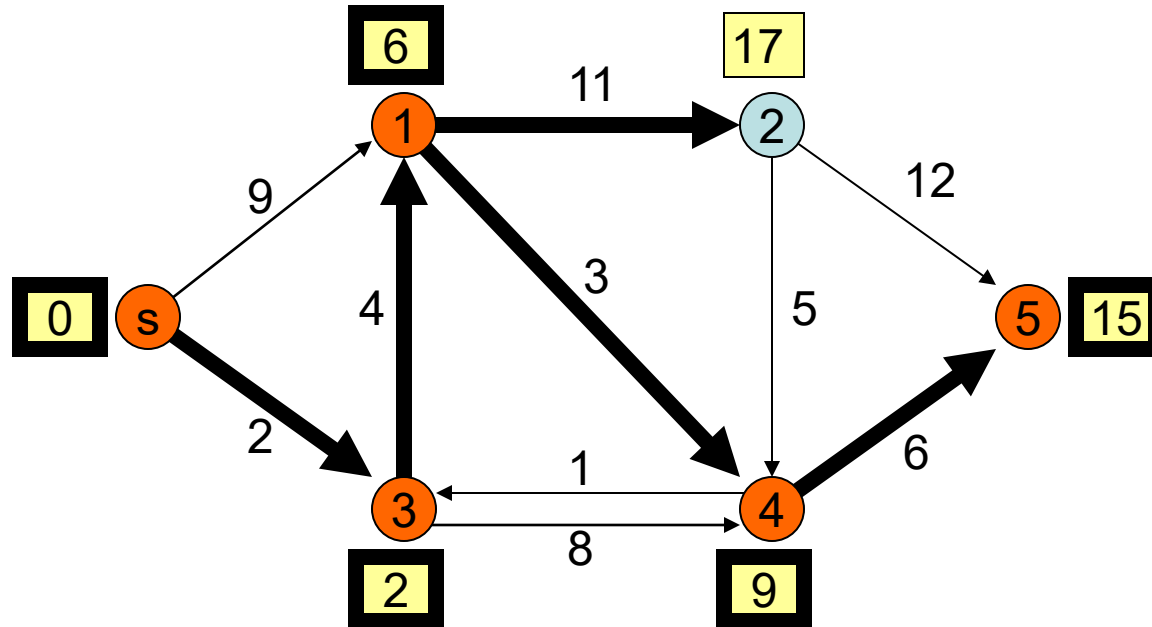
Examine node 1, which becomes permanently labeled.
Tighten all edges leaving node 1.

Dijkstra's Algorithm – Example



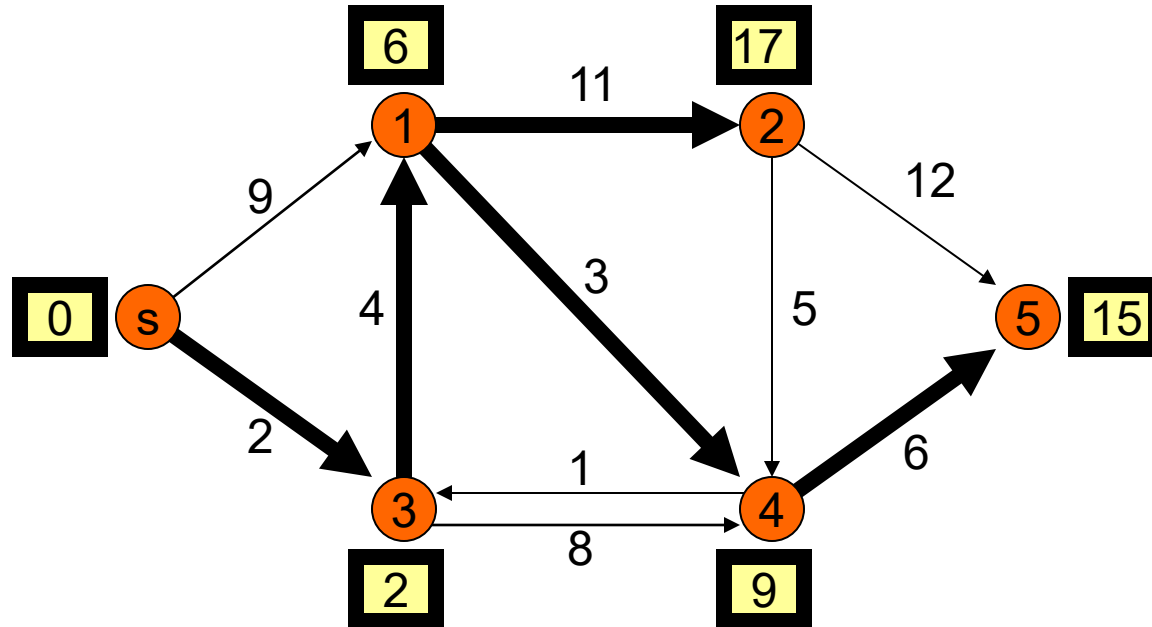
Examine node 4, which becomes permanently labeled.
Tighten all edges leaving node 4.

Dijkstra's Algorithm – Example



Examine node 5, which becomes permanently labeled.
Tighten all edges leaving node 5 (there aren't any).

Dijkstra's Algorithm – Example



Examine node 2, which becomes permanently labeled.
Algorithm terminates!

Dijkstra's Algorithm – Running Time

	Remove-Min $O(n)$ times	Decrease Key $O(n)$ times	Total Runtime
Unsorted Array	$O(n)$	$O(1)$	$O(n^2)$
Binary Heap	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Fibonacci Heap	$O(\log n)$	$O(1)$ amortized	$O(m + n \log n)$

- The Fibonacci heap bound is always at least as good as both other bounds in any graph; however, maybe not great in practice...
- In the RAM model (small integer edge costs), we can improve performance by using special RAM priority queues.
 - Running time here might depend on C , the magnitude of the largest edge cost.

Dijkstra's Algorithm – Correctness

- Just like BFS, Dijkstra visits all nodes in increasing order of distance from the source, and visits each node exactly once.
- **Claim:** when we visit node i , $c[i]$ will be the correct shortest path cost from s to i .
- Think of Dijkstra as maintaining two sets:
 - S: visited nodes that are “permanently” labeled.
 - T: remaining nodes that are temporarily labeled (whose cost labels are still upper bounds)
- Every iteration takes the node from T with minimum label and adds it to S.